

Largest Number:-

Code:-

```
class Solution {  
    public String largestNumber(int[] nums) {  
        String[] arr = new String[nums.length];  
        for (int i = 0; i < nums.length; i++) {  
            arr[i] = String.valueOf(nums[i]);  
        }  
        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));  
        if (arr[0].equals("0")) {  
            return "0";  
        }  
        StringBuilder ans = new StringBuilder();  
        for (String s : arr) {  
            ans.append(s);  
        }  
        return ans.toString();  
    }  
}
```

Output:-

The screenshot displays the LeetCode submission interface for the 'Largest Number' problem. The submission is marked as 'Accepted' with 237/237 testcases passed and a time taken of 6m 48s. The user 'tameem_shaik' submitted it on Sep 17, 2026 at 14:38. A promotional banner for 'Unlock the Full LeetCode Experience' is visible, offering a 56% discount on a ₹583.25/month subscription.

Runtime Performance:

- Runtime: 6 ms, Beats 93.18%
- Memory: 45.19 MB, Beats 29.84%

Test Results:

The 'Test Result' tab shows two test cases, both passed. Case 1 has an input of `nums = [10, 2]` and an output of `"210"`. Case 2 also has an output of `"210"`.

Code:

```
class Solution {  
    public String largestNumber(int[] nums) {  
        String[] arr = new String[nums.length];  
        for (int i = 0; i < nums.length; i++) {  
            arr[i] = String.valueOf(nums[i]);  
        }  
        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));  
        if (arr[0].equals("0")) {  
            return "0";  
        }  
        StringBuilder ans = new StringBuilder();  
        for (String s : arr) {  
            ans.append(s);  
        }  
        return ans.toString();  
    }  
}
```